

Labolatorium 7

April 29, 2026

Jakub Orchowski, s223281

1 CEL ĆWICZENIA

Konstrukcja baz funkcji B-sklejanych oraz porównanie krzywych 2D i 3D wyznaczanych metodą sumy funkcji bazowych i algorytmem de Boora.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
%matplotlib inline
```

2 Zadania

2.1 Zadanie 1.

Sprawdzam działanie funkcji bazowych stopnia 0 dla trzech zestawów węzłów podanych w instrukcji. Wariant (c) jest opcjonalny, ale został uwzględniony, żeby porównać wpływ losowego rozkładu węzłów na podpory funkcji bazowych.

```
[2]: T_VALUES = np.arange(0.0, 1.0001, 0.001)
RNG = np.random.default_rng(223281)

def bspline0(knots: np.ndarray, t_values: np.ndarray) -> np.ndarray:
    """Generuje unormowane funkcje bazowe stopnia 0 dla zadanego wektora węzłów.
    ↪ """
    basis = np.zeros((t_values.size, len(knots) - 1), dtype=np.float64)
    for index in range(len(knots) - 1):
        left, right = knots[index], knots[index + 1]
        if index == len(knots) - 2:
            mask = (t_values >= left) & (t_values <= right)
        else:
            mask = (t_values >= left) & (t_values < right)
        basis[mask, index] = 1.0
    return basis
```

```

def format_knots(knots: np.ndarray) -> str:
    """Zwraca czytelny tekst z wektorem węzłów."""
    return ', '.join(f'{value:.3f}' for value in knots)

knot_sets = {
    'Zestaw (a)': np.linspace(0.0, 1.0, 11),
    'Zestaw (b)': np.array([0.0, 0.1, 0.3, 0.4, 0.6, 0.65, 0.8, 0.9, 0.95, 1.0], dtype=np.float64),
    'Zestaw (c)': np.concatenate((
        [0.0],
        np.sort(RNG.uniform(0.05, 0.95, size=9)),
        [1.0],
    )),
}

basis0_sets = {label: bspline0(knots, T_VALUES) for label, knots in knot_sets.items()}

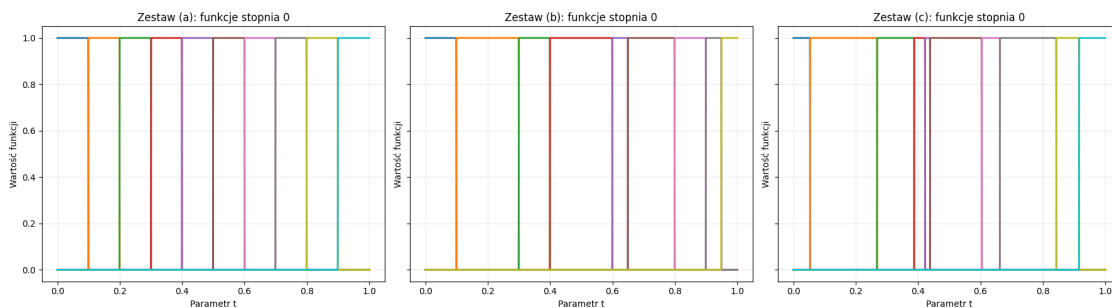
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

for ax, (label, knots) in zip(axes, knot_sets.items()):
    basis = basis0_sets[label]
    for column in range(basis.shape[1]):
        ax.plot(T_VALUES, basis[:, column], linewidth=2)
    ax.set_title(f'{label}: funkcje stopnia 0')
    ax.set_xlabel('Parametr t')
    ax.set_ylabel('Wartość funkcji')
    ax.grid(alpha=0.25)

plt.tight_layout()
plt.show()

print('Wektory węzłów użyte w dalszej części notebooka:')
for label, knots in knot_sets.items():
    print(f'{label}: {format_knots(knots)}')

```



Wektory węzłów użyte w dalszej części notebooka:

Zestaw (a): 0.000, 0.100, 0.200, 0.300, 0.400, 0.500, 0.600, 0.700, 0.800, 0.900, 1.000

Zestaw (b): 0.000, 0.100, 0.300, 0.400, 0.600, 0.650, 0.800, 0.900, 0.950, 1.000

Zestaw (c): 0.000, 0.054, 0.269, 0.388, 0.422, 0.438, 0.604, 0.662, 0.844, 0.916, 1.000

2.1.1 Wnioski

Dla funkcji stopnia 0 każda baza jest równa 1 tylko na swoim przedziale i 0 poza nim, więc wykresy bezpośrednio pokazują podział osi parametru przez węzły. Równomierny rozkład węzłów daje identyczne szerokości podpór, natomiast w wariantach nierównomiernych baza zagęszcza się tam, gdzie węzły leżą bliżej siebie.

2.2 Zadanie 2.

Tworzę funkcje `bspline1`, `bspline2` i `bspline3` korzystające z rekurencji Coxa-de Boora, czyli z wielomianów stopnia bezpośrednio niższego. Dla każdego zestawu węzłów porównuję przebiegi baz stopni 1, 2 i 3.

```
[3]: def elevate_bspline_basis(knots: np.ndarray, t_values: np.ndarray, lower_basis: np.ndarray, degree: int) -> np.ndarray:
    """Buduje funkcje bazowe wyższego stopnia z baz stopnia bezpośrednio niższego."""
    basis_count = len(knots) - degree - 1
    basis = np.zeros((t_values.size, basis_count), dtype=np.float64)
    for index in range(basis_count):
        left_denominator = knots[index + degree] - knots[index]
        right_denominator = knots[index + degree + 1] - knots[index + 1]
        if left_denominator > 0:
            basis[:, index] += ((t_values - knots[index]) / left_denominator) * lower_basis[:, index]
        if right_denominator > 0:
            basis[:, index] += ((knots[index + degree + 1] - t_values) / right_denominator) * lower_basis[:, index + 1]
    return basis

def bspline1(knots: np.ndarray, t_values: np.ndarray, poly0: np.ndarray | None) -> np.ndarray:
    """Generuje funkcje bazowe stopnia 1."""
    poly0 = bspline0(knots, t_values) if poly0 is None else poly0
    return elevate_bspline_basis(knots, t_values, poly0, degree=1)
```

```

def bspline2(knots: np.ndarray, t_values: np.ndarray, poly1: np.ndarray | None
↳ = None) -> np.ndarray:
    """Generuje funkcje bazowe stopnia 2."""
    poly1 = bspline1(knots, t_values) if poly1 is None else poly1
    return elevate_bspline_basis(knots, t_values, poly1, degree=2)

def bspline3(knots: np.ndarray, t_values: np.ndarray, poly2: np.ndarray | None
↳ = None) -> np.ndarray:
    """Generuje funkcje bazowe stopnia 3."""
    poly2 = bspline2(knots, t_values) if poly2 is None else poly2
    return elevate_bspline_basis(knots, t_values, poly2, degree=3)

def plot_basis_family(ax, t_values: np.ndarray, basis: np.ndarray, title: str)
↳ -> None:
    """Rysuje komplet funkcji bazowych na zadanej osi."""
    for column in range(basis.shape[1]):
        ax.plot(t_values, basis[:, column], linewidth=2)
    ax.set_title(title)
    ax.grid(alpha=0.25)

basis_sets_by_degree = {}
for label, knots in knot_sets.items():
    poly0 = bspline0(knots, T_VALUES)
    poly1 = bspline1(knots, T_VALUES, poly0)
    poly2 = bspline2(knots, T_VALUES, poly1)
    poly3 = bspline3(knots, T_VALUES, poly2)
    basis_sets_by_degree[label] = {0: poly0, 1: poly1, 2: poly2, 3: poly3}

fig, axes = plt.subplots(3, 3, figsize=(18, 12), sharex=True)

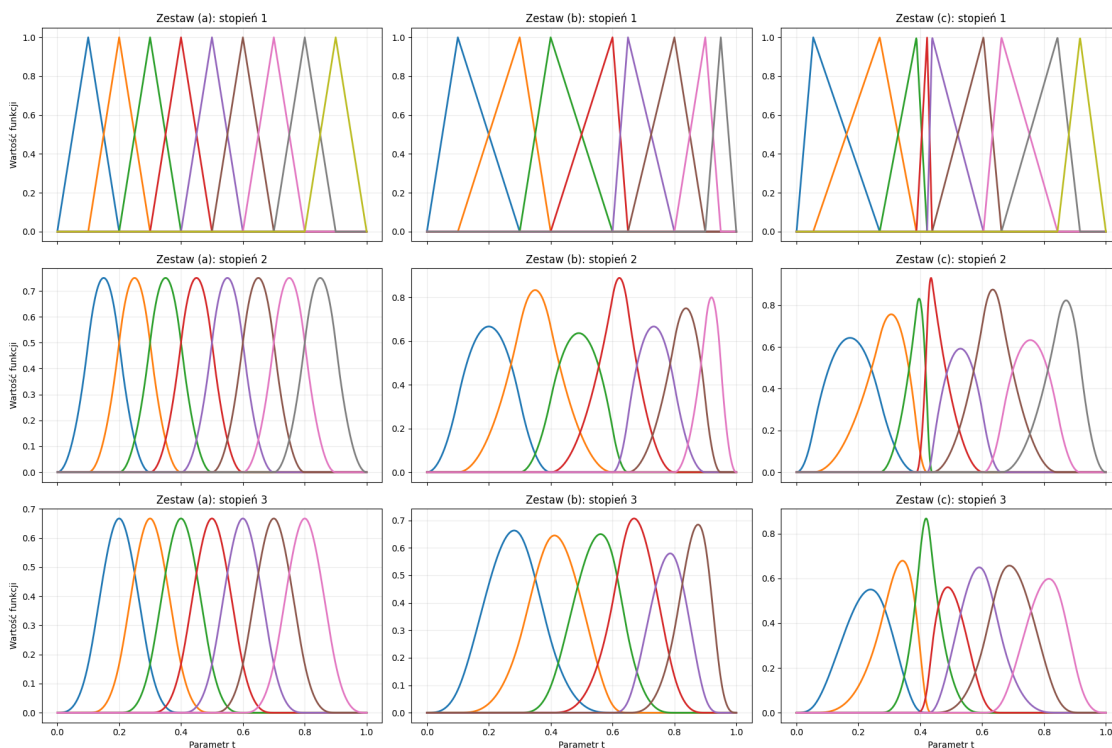
for row, degree in enumerate([1, 2, 3]):
    for col, label in enumerate(knot_sets):
        axis = axes[row, col]
        plot_basis_family(axis, T_VALUES, basis_sets_by_degree[label][degree],
↳ f'{label}: stopień {degree}')
        if row == 2:
            axis.set_xlabel('Parametr t')
        if col == 0:
            axis.set_ylabel('Wartość funkcji')

plt.tight_layout()
plt.show()

for label, degree_data in basis_sets_by_degree.items():

```

```
counts = ', '.join(f'stopień {degree}: {basis.shape[1]} funkcji' for
↳ degree, basis in degree_data.items())
print(f'{label}: {counts}')
```



Zestaw (a): stopień 0: 10 funkcji, stopień 1: 9 funkcji, stopień 2: 8 funkcji, stopień 3: 7 funkcji
 Zestaw (b): stopień 0: 9 funkcji, stopień 1: 8 funkcji, stopień 2: 7 funkcji, stopień 3: 6 funkcji
 Zestaw (c): stopień 0: 10 funkcji, stopień 1: 9 funkcji, stopień 2: 8 funkcji, stopień 3: 7 funkcji

2.2.1 Wnioski

Rekurencja Coxa-de Boora zwięża liczbę funkcji bazowych przy wzroście stopnia, ale jednocześnie wygładza ich kształt i poszerza podporę każdej z nich. Dla nierównomiernych węzłów maksima oraz szerokości podpór nie są symetryczne, więc lokalne zagęszczenie węzłów wyraźnie wpływa na przebieg bazy.

2.3 Zadanie 3.

Korzystam z baz stopnia 2, więc liczba punktów kontrolnych musi być równa liczbie funkcji bazowych, czyli $\text{len}(\text{knots}) - 3$. Krzywe 2D wyznaczam dwiema metodami: bezpośrednią sumą po funkcjach bazowych oraz algorytmem de Boora, a następnie porównuję obie postacie dla oryginalnych i zmodyfikowanych punktów kontrolnych.

```

[4]: CONTROL_POINTS_2D = np.array([
    [1.0, 0.0],
    [1.0, 1.0],
    [0.0, 1.0],
    [-1.0, 1.0],
    [-1.0, 0.0],
    [-1.0, -1.0],
    [0.0, -1.0],
    [1.0, -1.0],
], dtype=np.float64)
CONTROL_POINTS_2D_MODIFIED = CONTROL_POINTS_2D.copy()
CONTROL_POINTS_2D_MODIFIED[4] = np.array([0.0, 0.0])

def parameter_interval(knots: np.ndarray, degree: int, sample_count: int = 600) → np.ndarray:
    """Zwraca równomierny wektor parametrów na właściwym przedziale definicji krzywej."""
    basis_count = len(knots) - degree - 1
    start = knots[degree]
    stop = knots[basis_count]
    return np.linspace(start, stop, sample_count)

def evaluate_bspline_sum(knots: np.ndarray, control_points: np.ndarray, degree: int, parameter_values: np.ndarray) → np.ndarray:
    """Oblicza krzywą B-sklejaną ze wzoru sumującego funkcje bazowe."""
    poly0 = bspline0(knots, parameter_values)
    poly1 = bspline1(knots, parameter_values, poly0)
    poly2 = bspline2(knots, parameter_values, poly1)
    poly3 = bspline3(knots, parameter_values, poly2)
    basis_lookup = {0: poly0, 1: poly1, 2: poly2, 3: poly3}
    basis = basis_lookup[degree]
    return basis @ control_points

def de_boor_point(knots: np.ndarray, control_points: np.ndarray, degree: int, parameter: float) → np.ndarray:
    """Wyznacza pojedynczy punkt krzywej algorytmem de Boora."""
    control_points = np.asarray(control_points, dtype=np.float64)
    n = len(control_points) - 1
    knot_index = np.searchsorted(knots, parameter, side='right') - 1
    knot_index = min(max(knot_index, degree), n)
    local_points = [control_points[j + knot_index - degree].copy() for j in range(degree + 1)]

    for recursion_level in range(1, degree + 1):

```

```

        for j in range(degree, recursion_level - 1, -1):
            left_index = j + knot_index - degree
            right_index = j + 1 + knot_index - recursion_level
            denominator = knots[right_index] - knots[left_index]
            alpha = 0.0 if denominator == 0 else (parameter -
↪knots[left_index]) / denominator
            local_points[j] = (1.0 - alpha) * local_points[j - 1] + alpha *
↪local_points[j]
        return local_points[degree]

def evaluate_de_boor_curve(knots: np.ndarray, control_points: np.ndarray,
↪degree: int, parameter_values: np.ndarray) -> np.ndarray:
    """Wyznacza przebieg krzywej przez wielokrotne użycie algorytmu de Boora."""
    return np.vstack([de_boor_point(knots, control_points, degree, parameter)
↪for parameter in parameter_values])

def plot_bspline_case_2d(ax, knots: np.ndarray, control_points: np.ndarray,
↪title: str) -> float:
    """Rysuje krzywą 2D wyznaczoną dwiema metodami i zwraca maksymalną różnicę
↪między nimi."""
    parameter_values = parameter_interval(knots, degree=2)
    curve_sum = evaluate_bspline_sum(knots, control_points, degree=2,
↪parameter_values=parameter_values)
    curve_de_boor = evaluate_de_boor_curve(knots, control_points, degree=2,
↪parameter_values=parameter_values)
    max_difference = float(np.max(np.linalg.norm(curve_sum - curve_de_boor,
↪axis=1)))

    ax.plot(control_points[:, 0], control_points[:, 1], 'o--', color='0.55',
↪label='Wielokąt kontrolny')
    ax.plot(curve_sum[:, 0], curve_sum[:, 1], color='tab:blue', linewidth=2.5,
↪label='Wzór ze slajdu 38')
    ax.plot(curve_de_boor[:, 0], curve_de_boor[:, 1], color='tab:orange',
↪linestyle='--', linewidth=2.0, label='Wzór ze slajdu 42')
    ax.set_title(title)
    ax.axis('equal')
    ax.grid(alpha=0.25)
    return max_difference

curve_cases_2d = [
    ('Zestaw (a): punkty bazowe', knot_sets['Zestaw (a)'], CONTROL_POINTS_2D),
    ('Zestaw (a): po zmianie p4', knot_sets['Zestaw (a)'],
↪CONTROL_POINTS_2D_MODIFIED),

```

```

    ('Zestaw (b): bez punktu p7', knot_sets['Zestaw (b)'], CONTROL_POINTS_2D[:
↪-1]),
    ('Zestaw (c): punkty bazowe', knot_sets['Zestaw (c)'], CONTROL_POINTS_2D),
    ('Zestaw (c): po zmianie p4', knot_sets['Zestaw (c)'],
↪CONTROL_POINTS_2D_MODIFIED),
]

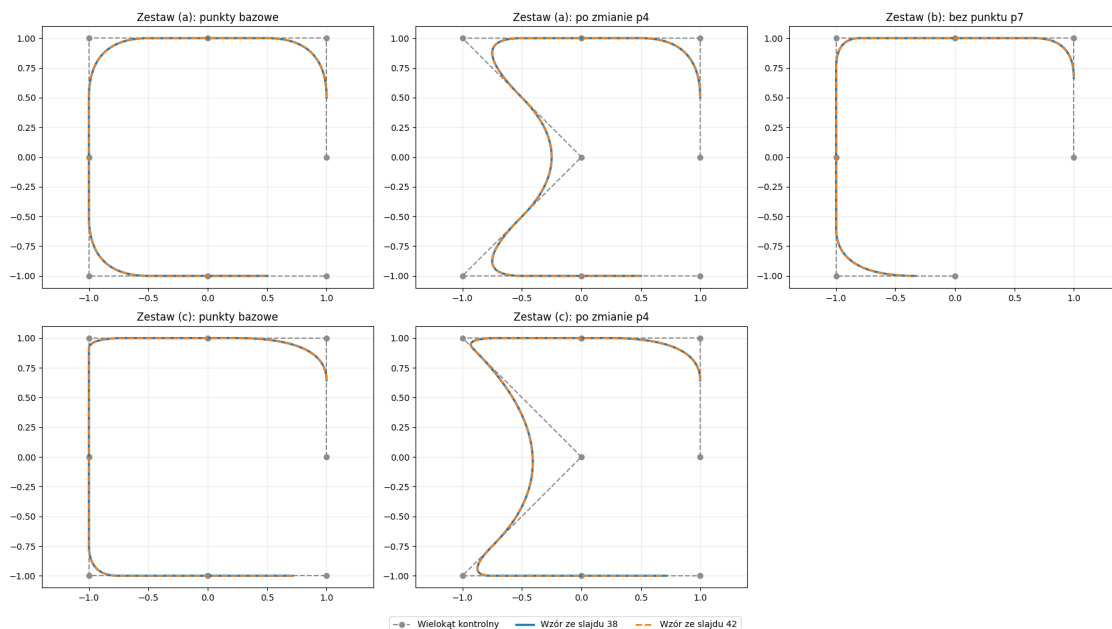
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
differences_2d = []

for axis, (title, knots, control_points) in zip(axes.flat, curve_cases_2d):
    difference = plot_bspline_case_2d(axis, knots, control_points, title)
    differences_2d.append((title, len(control_points), difference))

axes.flat[-1].axis('off')
handles, labels = axes.flat[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower center', ncol=3, bbox_to_anchor=(0.5, -0.
↪01))
plt.tight_layout(rect=(0.0, 0.03, 1.0, 1.0))
plt.show()

for title, point_count, difference in differences_2d:
    print(f'{title}: liczba punktów kontrolnych = {point_count}, maksymalna
↪różnica między metodami = {difference:.3e}')

```



Zestaw (a): punkty bazowe: liczba punktów kontrolnych = 8, maksymalna różnica między metodami = 3.140e-16

Zestaw (a): po zmianie p4: liczba punktów kontrolnych = 8, maksymalna różnica między metodami = 3.140e-16
Zestaw (b): bez punktu p7: liczba punktów kontrolnych = 7, maksymalna różnica między metodami = 3.140e-16
Zestaw (c): punkty bazowe: liczba punktów kontrolnych = 8, maksymalna różnica między metodami = 3.140e-16
Zestaw (c): po zmianie p4: liczba punktów kontrolnych = 8, maksymalna różnica między metodami = 3.140e-16

2.3.1 Wnioski

Dla stopnia 2 liczba punktów kontrolnych rzeczywiście musi odpowiadać liczbie funkcji bazowych, więc zestawy (a) i (c) używają 8 punktów, a zestaw (b) 7 punktów. Obie implementacje krzywej dają ten sam wynik z dokładnością numeryczną, a modyfikacja pojedynczego punktu kontrolnego wpływa lokalnie na kształt przebiegu dzięki własności lokalnego nośnika baz B-sklejanych.

2.4 Zadanie 4.

Tworzę przestrzenne krzywe B-sklejane drugiego stopnia dla punktów 3D z instrukcji i ponownie porównuję wzór sumacyjny z algorytmem de Boora. Dla zestawów (a) oraz (c) sprawdzam także wpływ przesunięcia punktu p3 na kształt przestrzennej trajektorii.

```
[5]: CONTROL_POINTS_3D = np.array([
    [0.0, 0.0, 0.0],
    [1.0, 0.0, 0.0],
    [1.0, 1.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 1.0, 1.0],
    [1.0, 1.0, 1.0],
    [1.0, 0.0, 1.0],
    [0.0, 0.0, 1.0],
], dtype=np.float64)
CONTROL_POINTS_3D_MODIFIED = CONTROL_POINTS_3D.copy()
CONTROL_POINTS_3D_MODIFIED[3] = np.array([0.5, 0.5, 0.0])

def set_equal_3d_axes(ax, points: np.ndarray) -> None:
    """Ustawia jednakową skalę na wszystkich osiach wykresu 3D."""
    min_corner = points.min(axis=0)
    max_corner = points.max(axis=0)
    center = 0.5 * (min_corner + max_corner)
    radius = 0.5 * np.max(max_corner - min_corner)
    ax.set_xlim(center[0] - radius, center[0] + radius)
    ax.set_ylim(center[1] - radius, center[1] + radius)
    ax.set_zlim(center[2] - radius, center[2] + radius)
```

```

def plot_bspline_case_3d(ax, knots: np.ndarray, control_points: np.ndarray,
    ↪title: str) -> float:
    """Rysuje przestrzenną krzywą 3D wyznaczoną dwiema metodami i zwraca
    ↪maksymalną różnicę między nimi."""
    parameter_values = parameter_interval(knots, degree=2)
    curve_sum = evaluate_bspline_sum(knots, control_points, degree=2,
    ↪parameter_values=parameter_values)
    curve_de_boor = evaluate_de_boor_curve(knots, control_points, degree=2,
    ↪parameter_values=parameter_values)
    max_difference = float(np.max(np.linalg.norm(curve_sum - curve_de_boor,
    ↪axis=1)))

    ax.plot(control_points[:, 0], control_points[:, 1], control_points[:, 2],
    ↪'o--', color='0.55', label='Wielokąt kontrolny')
    ax.plot(curve_sum[:, 0], curve_sum[:, 1], curve_sum[:, 2], color='tab:
    ↪blue', linewidth=2.5, label='Wzór ze slajdu 38')
    ax.plot(curve_de_boor[:, 0], curve_de_boor[:, 1], curve_de_boor[:, 2],
    ↪color='tab:orange', linestyle='--', linewidth=2.0, label='Wzór ze slajdu 42')
    ax.set_title(title)
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.view_init(elev=24, azim=-58)
    set_equal_3d_axes(ax, np.vstack((control_points, curve_sum, curve_de_boor)))
    return max_difference

curve_cases_3d = [
    ('Zestaw (a): punkty bazowe', knot_sets['Zestaw (a)'], CONTROL_POINTS_3D),
    ('Zestaw (a): po zmianie p3', knot_sets['Zestaw (a)'],
    ↪CONTROL_POINTS_3D_MODIFIED),
    ('Zestaw (c): punkty bazowe', knot_sets['Zestaw (c)'], CONTROL_POINTS_3D),
    ('Zestaw (c): po zmianie p3', knot_sets['Zestaw (c)'],
    ↪CONTROL_POINTS_3D_MODIFIED),
]

fig = plt.figure(figsize=(16, 12))
differences_3d = []

for subplot_index, (title, knots, control_points) in enumerate(curve_cases_3d,
    ↪start=1):
    axis = fig.add_subplot(2, 2, subplot_index, projection='3d')
    difference = plot_bspline_case_3d(axis, knots, control_points, title)
    differences_3d.append((title, difference))

handles, labels = fig.axes[0].get_legend_handles_labels()

```

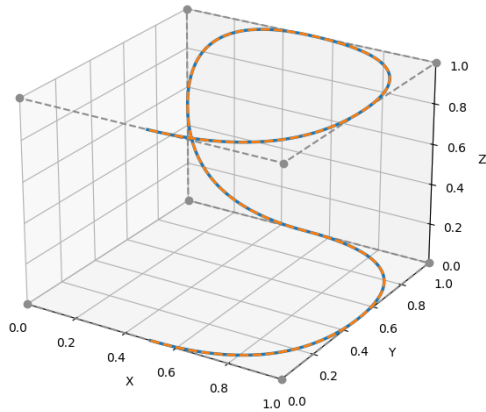
```

fig.legend(handles, labels, loc='lower center', ncol=3, bbox_to_anchor=(0.5, 0.
↪02))
plt.tight_layout(rect=(0.0, 0.05, 1.0, 1.0))
plt.show()

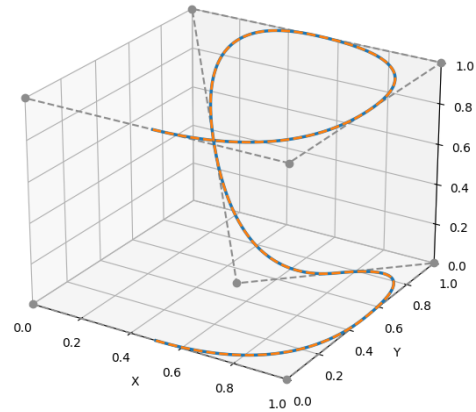
for title, difference in differences_3d:
    print(f'{title}: maksymalna różnica między metodami = {difference:.3e}')

```

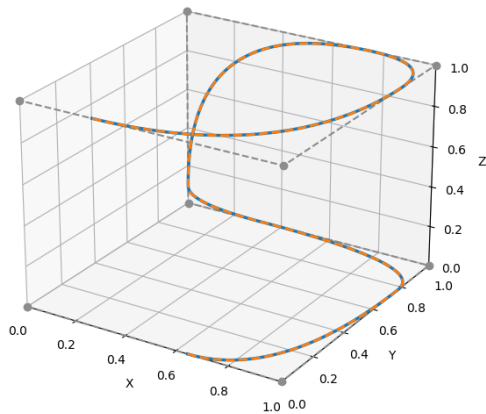
Zestaw (a): punkty bazowe



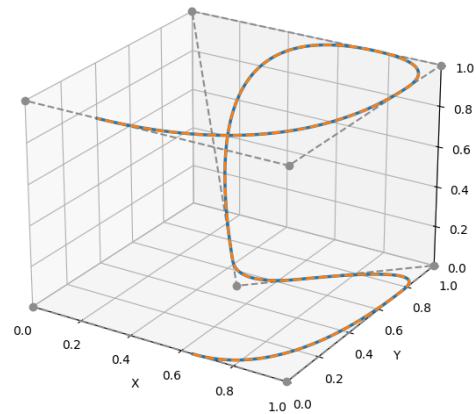
Zestaw (a): po zmianie p3



Zestaw (c): punkty bazowe



Zestaw (c): po zmianie p3



—●— Wielokąt kontrolny
 — Wzór ze slajdu 38
 - - - Wzór ze slajdu 42

Zestaw (a): punkty bazowe: maksymalna różnica między metodami = 3.846e-16
 Zestaw (a): po zmianie p3: maksymalna różnica między metodami = 3.846e-16
 Zestaw (c): punkty bazowe: maksymalna różnica między metodami = 3.846e-16
 Zestaw (c): po zmianie p3: maksymalna różnica między metodami = 3.846e-16

2.4.1 Wnioski

W przestrzeni 3D obie postacie krzywej ponownie pokrywają się numerycznie, co potwierdza poprawność implementacji baz oraz algorytmu de Boora. Przesunięcie punktu p_3 zmienia tylko lokalny fragment toru, a nierównomierny rozkład węzłów dodatkowo wpływa na tempo przesuwania się punktu po krzywej.